ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
DATA ENGINEERING (CO5173)
NHÓM 16

ỨNG DỤNG XỬ LÝ MICRO-BATCH TRONG GIÁM SÁT VÀ QUẢN LÝ NÔNG NGHIỆP

GVHD: PGS.TS. VÕ THỊ NGỌC CHÂU

Sinh viên:	Đoàn Trương Thanh Tịnh	2570761
	Nguyễn Hồ Tiến Đạt	2670265
	Lê Duy Khang	2670301
	Hoàng Chí Nhân	2670343

TP. Hồ Chí Minh, Ngày 13 tháng 11 năm 2026

LỜI CAM KẾT

Nhóm xin cam đoan đây là công trình của nhóm, được thực hiện dưới sự hướng dẫn của PGS.TS. Võ Thị Ngọc Châu. Mọi tài liệu và công cụ của bên thứ ba được sử dụng đều ghi rõ nguồn gốc.

Chữ kí GVHD

Trân trọng,

Nhóm 16

TÓM LƯỢC

[Tóm lược báo cáo: bối cảnh, giải pháp micro-batch, kết quả đánh giá chính.]

Từ khóa: Micro-batch, Spark Structured Streaming, Kafka, Nông nghiệp, Data Engineering.

Mục lục

Danh mục từ viết tắt **viii**

Chương 1 GIỚI THIỆU TỔNG QUAN **1**

1.1 Bối cảnh đề tài . . . 1
1.2 Mục tiêu của đề tài . . . 1
1.3 Phạm vi của đề tài . . . 1
1.4 Phân công công việc . . . 1
1.5 Bố cục báo cáo . . . 1

Chương 2 CƠ SỞ LÝ THUYẾT **2**

2.1 Thách thức của xử lý luồng thuần túy . . . 2
2.1.1 Khôi phục trạng thái đang xử lý . . . 2
2.1.2 Ghi trùng khi ghi dở dang . . . 2
2.1.3 Độ phức tạp trạng thái ở mức dưới một giây . . . 3
2.2 Khái niệm xử lý micro-batch . . . 3
2.3 Cơ chế hoạt động . . . 3
2.3.1 Kiến trúc pipeline . . . 3
2.3.2 Vòng đời xử lý của một micro-batch . . . 4
2.3.3 Chu kỳ trigger và ví dụ dữ liệu nông nghiệp . . . 4
2.3.4 Độ tin cậy: phục hồi sau lỗi và dữ liệu đến muộn . . . 4
2.4 Công nghệ hiện thực . . . 5
2.4.1 Apache Kafka làm nguồn dữ liệu . . . 5
2.4.2 Spark Structured Streaming . . . 5
2.4.3 Checkpoint . . . 6
2.4.4 Ví dụ mã nguồn . . . 6
2.5 So sánh, trường hợp sử dụng và đánh đổi . . . 7
2.5.1 So sánh ba mô hình xử lý . . . 7
2.5.2 Trường hợp sử dụng . . . 7
2.5.3 Được và mất . . . 7
2.5.4 Liên hệ với đề tài . . . 8
2.6 dbt và kỹ thuật bảo đảm chất lượng dữ liệu . . . 8
2.6.1 Kiến trúc medallion . . . 8
2.6.2 dbt: biến đổi dữ liệu dưới dạng mã . . . 8
2.6.3 Kiểm thử dữ liệu trong dbt . . . 9

2.6.4 Kiểm định, cách ly và gắn cờ . . . 10
2.6.5 Lưu vết nguồn gốc (lineage) . . . 10
2.6.6 Ghi idempotent và tính đúng đắn khi phát lại . . . 10
Chương 3 PHÂN TÍCH YÊU CẦU VÀ DỮ LIỆU 12
3.1 Lĩnh vực ứng dụng: giám sát nông nghiệp . . . 13
3.2 Các bên liên quan . . . 13
3.3 Yêu cầu nghiệp vụ . . . 13
3.4 Nhu cầu sử dụng dữ liệu . . . 14
3.4.1 Nhu cầu dữ liệu của BR4 và BR8 . . . 14
3.5 Nguồn dữ liệu và đặc trưng dữ liệu . . . 14
3.5.1 Nguồn dữ liệu . . . 14
3.5.2 Đặc trưng dữ liệu . . . 14
3.6 Ánh xạ yêu cầu nghiệp vụ sang yêu cầu xử lý dữ liệu . . . 14
3.6.1 Cách đọc ma trận . . . 15
3.6.2 Yêu cầu xử lý cho BR4 và BR8 . . . 15
Chương 4 THIẾT KẾ GIẢI PHÁP VÀ HIỆN THỰC ỨNG DỤNG 17
4.1 Kiến trúc tổng thể . . . 17
4.2 Công nghệ quản lý dữ liệu . . . 17
4.3 Công nghệ xử lý dữ liệu . . . 17
4.4 Thiết kế mô hình dữ liệu . . . 17
4.5 Pipeline micro-batch theo từng yêu cầu nghiệp vụ . . . 17
4.5.1 BR01 – Tập trung hóa giám sát nông nghiệp . . . 17
4.5.2 BR02 – Nâng cao nhận biết điều kiện đồng ruộng . . . 17
4.5.3 BR03 – Duy trì tính liên tục của giám sát . . . 17
4.5.4 BR04 – Thiết lập dữ liệu đáng tin cậy . . . 17
4.5.5 BR05 – Hỗ trợ quản lý dựa trên dữ liệu . . . 20
4.5.6 BR06 – Đánh giá lịch sử và lập kế hoạch . . . 20
4.5.7 BR07 – Làm cho thông tin dễ tiếp cận . . . 20
4.5.8 BR08 – Hỗ trợ ứng phó sớm với rủi ro . . . 20
4.6 Giải pháp thay thế để so sánh . . . 24
4.7 Hiện thực ứng dụng . . . 24
4.7.1 Mô phỏng và thu thập dữ liệu . . . 24
4.7.2 Giao diện người dùng . . . 24
4.7.3 Triển khai và cấu hình . . . 24
Chương 5 ĐÁNH GIÁ 27
5.1 Thiết lập thực nghiệm . . . 27
5.1.1 Phần cứng và phần mềm . . . 27
5.1.2 Dữ liệu thực nghiệm . . . 27
5.1.3 Tham số của đường xử lý . . . 29
5.1.4 Quy trình chạy lại . . . 29

5.2 Tính đúng đắn của dữ liệu 29
5.2.1 Phát hiện lỗi được tiêm vào 30
5.2.2 Đối soát tầng gold với tầng silver 32
5.2.3 Giá trị phi lý trong tầng gold 32
5.2.4 Không mất, không trùng khi khởi động lại 32
5.2.5 Kiểm thử đơn vị 33
5.3 Hiệu năng của giải pháp 33
5.4 Hiệu quả hỗ trợ khai thác dữ liệu qua ứng dụng 33
5.4.1 BR04 – Dữ liệu đáng tin cậy 33
5.4.2 BR08 – Ứng phó sớm với rủi ro 34
5.5 So sánh với giải pháp thay thế 35
5.6 Thảo luận 35
Chương 6 KẾT LUẬN **36**
6.1 Kết quả đạt được 36
6.2 Hạn chế 36
6.3 Hướng phát triển 36
Tài liệu tham khảo **37**

Danh sách hình ảnh

2.1 Rời rạc hóa luồng dữ liệu thành các micro-batch 3
2.2 Kiến trúc pipeline micro-batch từ cảm biến đến bảng điều khiển 4

4.1 Luồng bảo đảm chất lượng dữ liệu của BR04 18
4.2 Trang chất lượng dữ liệu của ứng dụng (ảnh chụp với dữ liệu mô phỏng của chế độ `DATA_MODE=mock`). 20
4.3 Trang cảnh báo của ứng dụng (ảnh chụp với dữ liệu mô phỏng của chế độ `DATA_MODE=mock`). 24

Danh sách bảng

2.1 Ví dụ hai micro-batch 5 giây của một cảm biến độ ẩm đất 4
2.2 So sánh xử lý theo lô, micro-batch và xử lý luồng 7
2.3 Lợi ích và chi phí của micro-batch 7

3.1 Danh sách yêu cầu nghiệp vụ (Business Requirements) 13
3.2 Nhu cầu sử dụng dữ liệu của BR4 và BR8 14
3.3 Ma trận truy vết giữa yêu cầu nghiệp vụ và thành phần xử lý dữ liệu . . . 15
3.4 Ánh xạ yêu cầu chức năng của BR4 và BR8 sang bước xử lý 16

4.1 Ngưỡng kiểm định theo chỉ số (`reference/reference_data.json`) 19
4.2 Các điều kiện cảnh báo (`src/streaming/pipeline.py`, `_build_conditions`) 21
4.3 Các bảng cảnh báo trong Cassandra (`cassandra/schema.cql`) 23
4.4 Các profile Docker Compose và dịch vụ (`docker-compose.yml`) 25
4.5 Phiên bản các thành phần 25
4.6 Các recipe `just` chính 26

5.1 Môi trường thực nghiệm 27
5.2 Cấu hình bộ mô phỏng (`.env.example`) 28
5.3 Tham số mặc định của tác vụ streaming và cảnh báo 29
5.4 Kết quả mong đợi cho từng loại lỗi được tiêm vào 30
5.5 Số lỗi tiêm vào so với số lỗi phát hiện được 31
5.6 Tiêu chí chấp nhận của BR4 và bằng chứng 33
5.7 Tiêu chí chấp nhận của BR8 và bằng chứng 34

Danh mục từ viết tắt

Viết tắt	Tiếng Anh	Tiếng Việt
BR	Business Requirement	Yêu cầu nghiệp vụ
ETL	Extract – Transform – Load	Trích xuất – biến đổi – nạp dữ liệu
IoT	Internet of Things	Internet vạn vật
LoRaWAN	Long Range Wide Area Network	Mạng diện rộng tầm xa

CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN

1.1 Bối cảnh đề tài

1.2 Mục tiêu của đề tài

1.3 Phạm vi của đề tài

1.4 Phân công công việc

1.5 Bố cục báo cáo

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết của kỹ thuật xử lý micro-batch, là kỹ thuật xử lý dữ liệu mà đề tài áp dụng cho lĩnh vực nông nghiệp. Mục 2.1 nêu các thách thức của xử lý luồng thuần túy, là động lực dẫn tới micro-batch. Mục 2.2 định nghĩa micro-batch. Mục 2.3 giải thích cơ chế hoạt động, từ kiến trúc, vòng đời một lô đến cách xử lý lỗi và dữ liệu đến muộn. Mục 2.4 giới thiệu bộ công nghệ Apache Kafka và Spark Structured Streaming. Mục 2.5 so sánh micro-batch với xử lý theo lô và xử lý luồng, cùng các đánh đổi.

2.1 Thách thức của xử lý luồng thuần túy

Trong xử lý luồng thuần túy (pure stream processing), mỗi sự kiện được xử lý ngay khi đến. Cách làm này cho độ trễ thấp, nhưng khi áp dụng cho dữ liệu cảm biến nông nghiệp thì nảy sinh ba khó khăn chính.

2.1.1 Khôi phục trạng thái đang xử lý

Khi một nút xử lý gặp sự cố giữa chừng, hệ thống phải dựng lại trạng thái phức tạp của hàng nghìn sự kiện chưa được đệm. Ví dụ, một nút bị lỗi khi đang nhận đợt dữ liệu dồn kéo dài 4 giờ từ cổng LoRaWAN sẽ rơi vào vòng lặp khởi động lại liên tục vì phải khôi phục dữ liệu đất chưa được đệm.

2.1.2 Ghi trùng khi ghi dở dang

Xử lý từng sự kiện không có ranh giới thực thi rõ ràng. Khi mạng đứt giữa chừng, việc tránh ghi trùng đòi hỏi cơ chế khóa phân tán tốn kém. Ví dụ, kết nối LoRaWAN đứt giữa lúc đồng bộ dữ liệu tưới khiến gói tin gửi lại bị cộng hai lần vào nhật ký lượng nước.

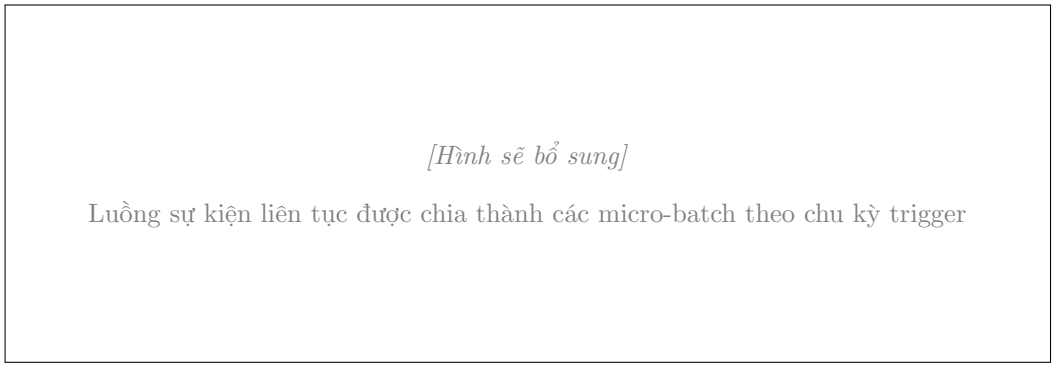
2.1.3 Độ phức tạp trạng thái ở mức dưới một giây

Xử lý luồng liên tục cần cơ chế watermark cho dữ liệu đến sai thứ tự và tiêu tốn nhiều bộ nhớ, làm tăng chi phí cho những tác vụ không cần độ trễ dưới một giây. Ví dụ, chạy vòng đánh giá dưới một giây trên nhiệt độ đất vốn biến thiên chậm là lãng phí, vì chu kỳ cập nhật 30 giây đã đáp ứng nhu cầu canh tác.

2.2 Khái niệm xử lý micro-batch

Micro-batch là mô hình xử lý lai: chia một luồng dữ liệu liên tục, không giới hạn thành chuỗi các khối dữ liệu nhỏ, hữu hạn (micro-batch), mỗi khối được thực thi sau một khoảng thời gian ngắn. Kleppmann mô tả ý tưởng này là "chia luồng thành các khối nhỏ và xử lý mỗi khối như một tiến trình xử lý theo lô thu nhỏ" [1].

Nhờ ranh giới rõ ràng của từng khối, mỗi lô có thể được ghi nhận, thử lại và cam kết (commit) như một đơn vị độc lập. Mô hình kết hợp độ trễ thấp của xử lý luồng với thông lượng cao và khả năng chịu lỗi của xử lý theo lô [2].

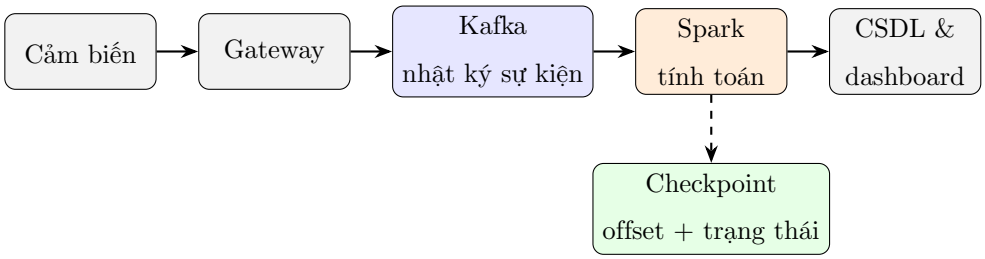


Hình 2.1: Rời rạc hóa luồng dữ liệu thành các micro-batch

2.3 Cơ chế hoạt động

2.3.1 Kiến trúc pipeline

Một pipeline micro-batch điển hình cho dữ liệu cảm biến gồm các thành phần nối tiếp trong Hình 2.2. Dữ liệu từ cảm biến qua cổng kết nối (gateway) vào nhật ký sự kiện Kafka. Kafka tiếp tục nhận sự kiện trong khi Spark đang tính toán, nhờ đó khâu thu thập và khâu xử lý tách rời nhau. Kết quả được ghi vào cơ sở dữ liệu phục vụ bảng điều khiển. Tiến độ xử lý và trạng thái được lưu bền vững trong checkpoint.



Hình 2.2: Kiến trúc pipeline micro-batch từ cảm biến đến bảng điều khiển

2.3.2 Vòng đời xử lý của một micro-batch

Mỗi micro-batch trải qua sáu bước:

1. **Trigger**: bắt đầu lượt chạy tiếp theo.
2. **Lập kế hoạch và ghi offset**: chọn các bản ghi mới và ghi lại phạm vi offset sẽ đọc.
3. **Đọc và biến đổi**: áp dụng truy vấn lên dữ liệu của lô.
4. **Cập nhật trạng thái**: dành cho các truy vấn có trạng thái (cửa sổ, tổng hợp, khử trùng lặp).
5. **Ghi kết quả**: gửi đầu ra tới nơi lưu (sink).
6. **Cam kết lô**: ghi nhận lô đã hoàn tất.

2.3.3 Chu kỳ trigger và ví dụ dữ liệu nông nghiệp

Chu kỳ trigger quyết định độ chi tiết của lô. Giả sử một cảm biến độ ẩm đất gửi số đo lên Kafka, và trigger chạy mỗi 5 giây (Bảng 2.1). Mỗi lô tính một giá trị tổng hợp rồi lưu kết quả, hiển thị trên bảng điều khiển và đưa vào các quy tắc cảnh báo.

Bảng 2.1: Ví dụ hai micro-batch 5 giây của một cảm biến độ ẩm đất

Lô	**Trigger**	**Số đo đến Kafka (độ ẩm %)**	**Trung bình**
1	08:00:05	22 (08:00:01), 24 (08:00:03), 26 (08:00:04)	24%
2	08:00:10	28 (08:00:07), 30 (08:00:09)	29%

2.3.4 Độ tin cậy: phục hồi sau lỗi và dữ liệu đến muộn

Phục hồi sau lỗi. Checkpoint bền vững lưu offset và trạng thái truy vấn. Khi khởi động lại, hệ thống phát lại và hoàn tất lô chưa được cam kết, nên không mất và không bỏ sót dữ liệu.

Dữ liệu đến muộn. Cần phân biệt thời gian sự kiện (event time) với thời gian đến (arrival time). Một số đo có thể được tạo lúc 08:00:03 nhưng đến hệ thống sau đó vì mất kết nối. Hệ thống giữ trạng thái cửa sổ để nhận các số đo muộn này. *Watermark* đi theo tiến độ của thời gian sự kiện; nó không phải bộ đếm chờ theo đồng hồ thực. Khi watermark tiến lên, trạng thái của các cửa sổ đủ điều kiện được dọn, giữ bộ nhớ ở mức giới hạn.

Điều kiện exactly-once. Kết quả "đúng một lần" (exactly-once) cần đồng thời ba điều kiện: nguồn có thể phát lại (replayable), engine có checkpoint, và nơi ghi an toàn khi thử lại (idempotent hoặc transactional). Nếu chỉ dùng sink Kafka thông thường thì chỉ đạt at-least-once.

2.4 Công nghệ hiện thực

Bộ ba công nghệ được dùng để hiện thực micro-batch gồm Apache Kafka làm nguồn, Spark Structured Streaming làm engine xử lý và checkpoint bảo đảm độ tin cậy.

2.4.1 Apache Kafka làm nguồn dữ liệu

Kafka là nhật ký sự kiện bền vững và có thể phát lại [3]. Dữ liệu được tổ chức theo *topic*, mỗi topic chia thành các *partition*; mỗi thông điệp trong partition có một *offset* tăng dần. Thông điệp được giữ trong một thời gian lưu trữ (retention) kể cả sau khi đã đọc, nên Spark có thể đọc lại bất kỳ khoảng offset nào. Kafka cũng tách bên sinh dữ liệu khỏi bên xử lý. Một partition Kafka tương ứng một tác vụ (task) Spark.

Ở đầu mỗi lô, Spark hỏi Kafka offset mới nhất của từng partition, rồi xác định khoảng offset cần đọc (ví dụ partition 0: 1000–1500, partition 1: 980–1420). Khoảng offset này chính là micro-batch. Các partition được đọc song song, và offset được theo dõi bằng checkpoint của Spark chứ không dùng consumer group. Tùy chọn `maxOffsetsPerTrigger` giới hạn số bản ghi mỗi lô, bảo vệ tác vụ khỏi các đợt tăng đột biến.

2.4.2 Spark Structured Streaming

Spark Structured Streaming coi luồng là một *bảng không giới hạn*: mỗi bản ghi mới là một dòng được nối vào bảng đầu vào. Ở mỗi trigger, Spark chạy truy vấn chỉ trên các dòng mới và cập nhật kết quả [4]. Nhờ đó cùng một API DataFrame/SQL dùng được cho cả xử lý theo lô và xử lý luồng. Ba khái niệm cấu hình chính:

- **Trigger** (khi nào chạy): `processingTime` chạy theo chu kỳ cố định, ví dụ mỗi 10 giây; mặc định là các lô nối tiếp nhau; `availableNow` xử lý hết dữ liệu hiện có rồi dừng.
- **Output mode** (ghi gì): *append* chỉ ghi dòng mới, *update* chỉ ghi dòng thay đổi, *complete* ghi lại toàn bộ bảng kết quả.

- **Watermark** (xử lý dữ liệu muộn): ví dụ "chờ tối đa 2 phút rồi chốt cửa sổ"; trạng thái cũ bị xóa nên bộ nhớ không tăng vô hạn.

2.4.3 Checkpoint

Thư mục checkpoint lưu bốn thành phần:

- `offsets/`: nhật ký ghi trước (write-ahead log). *Trước* khi lô N chạy, Spark ghi phạm vi offset sẽ đọc.
- `commits/`: *sau* khi lô N được ghi vào sink, Spark đánh dấu lô đã xong.
- `state/`: kho trạng thái cho cửa sổ, tổng hợp và khử trùng lặp.
- `metadata`: mã định danh duy nhất của truy vấn. Khi đổi lớn logic truy vấn, cần dùng vị trí checkpoint mới.

Quy trình phục hồi như sau: ghi offset vào `offsets/`, xử lý và ghi vào sink, rồi ghi `commits/`. Nếu hệ thống sập khi lô N đã có trong `offsets/` nhưng chưa có trong `commits/`, lô N được phát lại với *cùng* khoảng offset. Để kết quả đúng một lần, sink phải idempotent hoặc transactional, ví dụ Delta Lake hoặc upsert theo mã lô.

2.4.4 Ví dụ mã nguồn

Đoạn mã dưới đây đọc luồng đơn hàng từ Kafka, tính doanh thu theo sản phẩm trong cửa sổ 1 phút và cập nhật kết quả mỗi 10 giây. Cùng mẫu này sẽ được áp dụng cho số đo cảm biến trong Chương 4.

```
orders = (spark.readStream.format("kafka")
    .option("subscribe", "orders")
    .option("maxOffsetsPerTrigger", 10000)
    .load())

revenue = (orders
    .select(from_json("value", schema).alias("o")).select("o.*")
    .withWatermark("event_time", "2 minutes")
    .groupBy(window("event_time", "1 minute"), "product")
    .agg(sum("amount")))

(revenue.writeStream
    .outputMode("update")
    .trigger(processingTime="10 seconds")
    .option("checkpointLocation", "/tmp/chk/orders")
    .start())
```

Đoạn mã 2.1: Kafka → Spark → sink với trigger 10 giây

2.5 So sánh, trường hợp sử dụng và đánh đổi

2.5.1 So sánh ba mô hình xử lý

Bảng 2.2: So sánh xử lý theo lô, micro-batch và xử lý luồng

Tiêu chí	**Xử lý theo lô**	**Micro-batch**	**Xử lý luồng**
Đơn vị xử lý	Khối dữ liệu lớn	Lô nhỏ, thường xuyên	Từng sự kiện liên tục
Độ trễ	Phút đến giờ	Giây đến phút	Mili giây đến giây
Thông lượng	Cao, hiệu quả	Tốt, độ phức tạp vừa phải	Phụ thuộc thiết kế
Độ phức tạp	Đơn giản	Cân bằng độ trễ và hiệu quả	Cao hơn, chi phí vận hành lớn
Phù hợp với	ETL, báo cáo, phân tích lịch sử	Dashboard, ETL tăng dần, phân tích gần thời gian thực	Phát hiện gian lận, cảnh báo, quyết định thời gian thực

2.5.2 Trường hợp sử dụng

Micro-batch thường dùng cho: phân tích gần thời gian thực, xử lý nhật ký, giám sát IoT, phát hiện gian lận, gợi ý và cá nhân hóa, và các pipeline học máy.

2.5.3 Được và mất

Bảng 2.3: Lợi ích và chi phí của micro-batch

Lợi ích	**Chi phí**
Dữ liệu mới trong vài phút với công cụ theo lô quen thuộc	Độ trễ không thể thấp hơn chu kỳ lô
Thử lại dễ dàng, cam kết idempotent và transactional	Cam kết thường xuyên tạo nhiều tệp nhỏ và snapshot, cần gộp tệp (compaction) và dọn snapshot
Rẻ và đơn giản hơn xử lý luồng thực sự	Chu kỳ càng ngắn thì chi phí lập lịch và siêu dữ liệu càng cao

2.5.4 Liên hệ với đề tài

Dữ liệu nông nghiệp như độ ẩm đất hay nhiệt độ biến thiên chậm, kết nối LoRaWAN dễ gián đoạn, và yêu cầu nghiệp vụ thường chấp nhận độ trễ từ vài giây đến vài chục giây. Các đặc điểm này phù hợp với micro-batch: tránh chi phí của xử lý luồng thuần túy mà vẫn bảo đảm tính liên tục và đúng đắn của dữ liệu. Việc áp dụng cụ thể được trình bày ở Chương 4.

2.6 dbt và kỹ thuật bảo đảm chất lượng dữ liệu

Phần trên giải thích cách dữ liệu được đưa vào hệ thống theo từng micro-batch. Mục này trình bày nhóm kỹ thuật bảo đảm dữ liệu đó *đáng tin cậy*: kiến trúc phân tầng medallion, công cụ biến đổi dbt, cách kiểm định, cách ly bản ghi lỗi và lưu vết nguồn gốc (lineage). Đây là cơ sở cho yêu cầu BR4 và cho phần đánh giá tính đúng đắn ở Chương 5.

2.6.1 Kiến trúc medallion

Kiến trúc medallion chia kho dữ liệu thành ba tầng, mỗi tầng có một cam kết chất lượng rõ ràng [5]:

- **Bronze**: dữ liệu thô gần với nguồn, chỉ bổ sung siêu dữ liệu (thời điểm nhận, nguồn, cờ chất lượng). Tầng này giữ toàn bộ lịch sử để có thể xử lý lại.
- **Silver**: dữ liệu đã làm sạch, khử trùng lặp và chuẩn hóa kiểu. Mỗi sự kiện chỉ xuất hiện một lần.
- **Gold**: dữ liệu tổng hợp theo nhu cầu nghiệp vụ (theo ngày, theo thửa ruộng), sẵn sàng cho bảng điều khiển và phân tích.

Ưu điểm chính là *tách bạch trách nhiệm*: lỗi ở tầng sau luôn có thể sửa bằng cách tính lại từ tầng trước, vì tầng bronze không bao giờ bị ghi đè bằng dữ liệu đã biến đổi. Trong đề tài, bronze được ghi bởi tác vụ Spark Structured Streaming, còn silver và gold được dựng bằng dbt.

2.6.2 dbt: biến đổi dữ liệu dưới dạng mã

dbt (data build tool) cho phép viết mỗi bước biến đổi thành một *model*, tức một câu lệnh `SELECT` lưu trong tệp `.sql` [6]. dbt tự sinh câu lệnh tạo bảng, tự xác định thứ tự chạy nhờ hàm `ref()` và `source()`, và chạy câu lệnh trên một engine có sẵn. Đề tài dùng bộ chuyển đổi dbt-spark kết nối tới Spark Thrift Server, nên Spark vẫn là engine thực thi và các bảng đích là bảng Iceberg.

Đoạn mã 2.2 là model silver của đề tài. Hàm cửa sổ `row_number()` giữ lại đúng một bản ghi cho mỗi khóa `(device_id, metric, measured_at)`, ưu tiên bản ghi được nhận sau cùng.

```sql
{{ config(materialized='table') }}
with ranked as (
    select *,
        row_number() over (
            partition by device_id, metric, measured_at
            order by ingested_at desc, source_offset desc
        ) as row_num
    from {{ source('bronze', 'sensor_readings') }}
    where measured_at is not null
      and device_id is not null and metric is not null
)
select event_id, device_id, field_id, farm_id, crop_code, metric,
       value, unit, measured_at, ingested_at, is_late, quality_flag,
       source_topic, source_partition, source_offset, batch_id
from ranked
where row_num = 1
```

Đoạn mã 2.2: Model silver khử trùng lặp (`dbt/models/silver/sensor_readings.sql`, rút gọn)

2.6.3 Kiểm thử dữ liệu trong dbt

Mỗi model khai báo các *test* trong tệp `schema.yml`. Một test là một câu truy vấn trả về các dòng vi phạm; test đạt khi kết quả rỗng. dbt cung cấp sẵn bốn test chung: `not_null`, `unique`, `accepted_values` và `relationships`. Đoạn mã 2.3 là phần khai báo test của model silver.

```yaml
models:
  - name: sensor_readings
    tests:
      - unique:
          column_name: "concat(device_id, '|', metric, '|', cast(measured_at as
    string))"
    columns:
      - name: device_id
        tests: [not_null]
      - name: metric
        tests:
          - not_null
          - accepted_values:
              values: [soil_moisture, soil_temperature, air_temperature,
                       air_humidity, relative_humidity, rainfall]
```

Đoạn mã 2.3: Test của model silver (`dbt/models/silver/schema.yml`, rút gọn)

Lệnh `dbt build` chạy model và test theo đúng thứ tự phụ thuộc. Khi một test của model silver thất bại, các model gold phía sau bị bỏ qua (*skip*) thay vì được dựng từ dữ

liệu sai. Đây là cơ chế đề tài dùng để đáp ứng FR4.7: một test thất bại sẽ chặn lần làm mới tầng gold tương ứng.

2.6.4 Kiểm định, cách ly và gắn cờ

Không phải lỗi nào cũng nên xử lý giống nhau. Đề tài phân biệt hai mức:

- **Lỗi cấu trúc** (payload không phải JSON, thiếu trường bắt buộc, thời gian không đọc được, giá trị không phải số, thiết bị không có trong danh mục): bản ghi không thể gán cho thửa ruộng nào, nên được *cách ly* (quarantine) sang một bảng riêng kèm lý do và payload gốc. Tác vụ không dừng lại vì một bản ghi hỏng.
- **Lỗi nội dung** (giá trị rỗng, ngoài khoảng vật lý, bước nhảy bất thường, bản ghi trùng): bản ghi vẫn được lưu nhưng mang *cờ chất lượng* (`quality_flag`), để bảng điều khiển có thể loại bỏ hoặc đánh dấu.

Cách làm này giữ được toàn bộ bằng chứng: không có bản ghi nào bị xóa âm thầm, và số lượng của từng loại lỗi có thể đếm lại. Đoạn mã 2.4 là cấu trúc bảng cách ly.

```
CREATE TABLE IF NOT EXISTS lakehouse.ops.quarantine (
    reason string NOT NULL,       -- invalid_json | missing_field | ...
    raw_payload string,           -- original payload, unchanged
    device_id string, metric string, reading_ts string, value_raw string,
    source_topic string NOT NULL, source_partition int NOT NULL,
    source_offset bigint NOT NULL,
    ingested_at timestamp, detected_at timestamp, batch_id bigint
) USING iceberg PARTITIONED BY (days(detected_at))
```

Đoạn mã 2.4: Bảng cách ly trên Iceberg (`src/streaming/ddl.py`, rút gọn)

2.6.5 Lưu vết nguồn gốc (lineage)

Lineage là khả năng trả lời câu hỏi "con số này đến từ đâu". Ở mức bản ghi, đề tài gắn vào mọi dòng các cột sau: chủ đề, phân vùng và offset Kafka (`source_topic`, `source_partition`, `source_offset`), thời điểm Kafka nhận sự kiện (`ingested_at`), mã micro-batch đã ghi dòng đó (`batch_id`) và cờ chất lượng. Vì offset Kafka là duy nhất trong một phân vùng, bộ ba (topic, partition, offset) định danh chính xác một sự kiện gốc. Kết hợp với khả năng phát lại của Kafka (Mục 2.4.1), mọi giá trị trên bảng điều khiển đều truy ngược được tới sự kiện và lô đã tạo ra nó.

2.6.6 Ghi idempotent và tính đúng đắn khi phát lại

Mục 2.4.3 cho thấy một lô có thể được chạy lại sau sự cố. Để kết quả không bị nhân đôi, mọi lệnh ghi phải *idempotent*: chạy một lần hay nhiều lần cho cùng kết quả. Đề

tài dùng lệnh `MERGE INTO` của Iceberg với khóa tự nhiên của từng bảng, như trong Đoạn mã 2.5.

```
MERGE INTO lakehouse.bronze.sensor_readings AS t
USING src_bronze AS s
ON t.device_id = s.device_id AND t.metric = s.metric
   AND t.measured_at = s.measured_at
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Đoạn mã 2.5: Ghi idempotent bằng MERGE (`src/streaming/ddl.py`)

Khóa của bảng cách ly là (topic, partition, offset); khóa của bảng đếm chất lượng là (`batch_id`, `device_id`, `metric`). Cột `detected_at` trong bảng cách ly lấy bằng `ingested_at` chứ không lấy giờ hệ thống, để lần phát lại sinh ra dòng giống hệt lần đầu.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ DỮ LIỆU

3.1 Lĩnh vực ứng dụng: giám sát nông nghiệp

3.2 Các bên liên quan

3.3 Yêu cầu nghiệp vụ

Bảng 3.1: Danh sách yêu cầu nghiệp vụ (Business Requirements)

ID	Yêu cầu	Mô tả	Phụ trách
BR01	Tập trung hóa giám sát nông nghiệp	Cung cấp góc nhìn hợp nhất về trang trại, thửa ruộng và thiết bị giám sát để quản lý nhất quán trên nhiều địa điểm.	Đạt
BR02	Nâng cao nhận biết điều kiện đồng ruộng	Cung cấp kịp thời thông tin về môi trường và điều kiện đồng ruộng, giảm phụ thuộc vào đo thủ công.	[...]
BR03	Duy trì tính liên tục của giám sát	Giảm thiểu khoảng trống dữ liệu và bảo toàn thông tin khi thiết bị, kết nối hoặc dịch vụ gián đoạn tạm thời.	Nhân
BR04	Thiết lập dữ liệu nông nghiệp đáng tin cậy	Bảo đảm thông tin nhất quán, tin cậy, truy vết được; giảm rủi ro do số đo không hợp lệ, trùng lặp hoặc gây hiểu nhầm.	Tịnh
BR05	Hỗ trợ quản lý trang trại dựa trên dữ liệu	Dùng số đo và xu hướng để định hướng tưới tiêu, phân bổ tài nguyên và quản lý thửa ruộng, giảm lãng phí.	Tịnh
BR06	Đánh giá lịch sử và lập kế hoạch	Lưu trữ lịch sử để so sánh giữa các giai đoạn, nhận diện mùa vụ, đánh giá thực hành quản lý và lập kế hoạch.	[...]
BR07	Làm cho thông tin dễ tiếp cận	Cung cấp tóm tắt, xu hướng và so sánh thửa ruộng dễ hiểu cho quản lý và nhà phân tích.	Duy

3.4 Nhu cầu sử dụng dữ liệu

3.4.1 Nhu cầu dữ liệu của BR4 và BR8

BR4 và BR8 có nhu cầu dữ liệu khác nhau về độ trễ. BR4 cần một bức tranh đầy đủ và kiểm chứng được, nên chấp nhận độ trễ của một lần làm mới tầng gold. BR8 cần phát hiện sự cố sớm, nên phải chạy ngay trong từng micro-batch. Bảng 3.2 tóm tắt hai nhu cầu này.

Bảng 3.2: Nhu cầu sử dụng dữ liệu của BR4 và BR8

BR	**Người dùng**	**Câu hỏi cần trả lời**	**Loại xử lý**	**Độ trễ chấp nhận**
BR4	U4 nhóm dữ liệu, U3 nhà phân tích	Mỗi ngày có bao nhiêu bản ghi hợp lệ, trùng, ngoài khoảng, bị cách ly? Một giá trị cụ thể đến từ sự kiện và lô nào?	Kiểm định trong stream, đếm theo lô, tổng hợp theo ngày bằng dbt	Một lần làm mới gold (theo giờ hoặc theo ngày)
BR8	U2 vận hành viên, U1 quản lý	Thửa ruộng nào đang có điều kiện bất thường? Thiết bị nào mất kết nối? Ai đã xử lý cảnh báo, khi nào?	Luật đánh giá trên từng micro-batch, trạng thái cảnh báo trong Cassandra	Một đến hai chu kỳ trigger cộng thời gian debounce (khoảng 1–2 phút)

3.5 Nguồn dữ liệu và đặc trưng dữ liệu

3.5.1 Nguồn dữ liệu

3.5.2 Đặc trưng dữ liệu

3.6 Ánh xạ yêu cầu nghiệp vụ sang yêu cầu xử lý dữ liệu

Mục này nối các yêu cầu nghiệp vụ ở Mục 3.3 với các thành phần xử lý dữ liệu sẽ được thiết kế ở Chương 4. Bảng 3.3 là ma trận truy vết: một dấu ✓ nghĩa là thành phần đó tham gia trực tiếp vào việc đáp ứng yêu cầu.

Bảng 3.3: Ma trận truy vết giữa yêu cầu nghiệp vụ và thành phần xử lý dữ liệu

Yêu cầu	Kafka	Spark stream	Cassandı	Iceberg (bronze/gc	dbt	Trino	FastAPI / web
BR1 Tập trung hóa giám sát	✓	✓	✓	✓		✓	✓
BR2 Nhận biết điều kiện đồng ruộng	✓	✓	✓				✓
BR3 Liên tục giám sát	✓	✓	✓	✓	✓		
BR4 Dữ liệu đáng tin cậy		✓	✓	✓	✓	✓	✓
BR5 Quản lý dựa trên dữ liệu				✓	✓	✓	✓
BR6 Đánh giá lịch sử				✓	✓	✓	✓
BR7 Thông tin dễ tiếp cận			✓	✓		✓	✓
BR8 Ứng phó sớm với rủi ro	✓	✓	✓	✓	✓	✓	✓

3.6.1 Cách đọc ma trận

Ma trận cho thấy ba nhóm yêu cầu, tương ứng với ba đường dữ liệu của hệ thống:

- **Đường gần thời gian thực** (Kafka, Spark streaming, Cassandra): BR2 và BR3 dựa chủ yếu vào đường này, vì chúng cần dữ liệu mới trong khoảng một phút và cần chịu được gián đoạn.
- **Đường phân tích** (Iceberg, dbt, Trino): BR5 và BR6 chỉ dùng đường này, vì chúng làm việc trên dữ liệu tổng hợp nhiều ngày đến nhiều tháng.
- **Yêu cầu xuyên suốt**: BR1, BR4 và BR8 đi qua cả hai đường. BR8 chạm vào mọi thành phần: luật cảnh báo chạy trong Spark, trạng thái cảnh báo nằm trong Cassandra, còn lịch sử cảnh báo và dữ liệu thô đi vào Iceberg để phân tích sau.

3.6.2 Yêu cầu xử lý cho BR4 và BR8

Bảng 3.4 chi tiết hóa hai yêu cầu xuyên suốt BR4 và BR8 thành các yêu cầu chức năng (FR) và nơi chúng được hiện thực.

Bảng 3.4: Ánh xạ yêu cầu chức năng của BR4 và BR8 sang bước xử lý

FR	Yêu cầu	Thành phần	Cách hiện thực
FR4.1	Schema tường minh, cách ly bản ghi lỗi	Spark streaming	`StructType` tường minh; bảng `ops.quarantine`
FR4.2	Kiểm tra khoảng giá trị, phát hiện bước nhảy	Spark streaming	Ngưỡng `min`, `max`, `max_jump` trong dữ liệu danh mục
FR4.3	Khử trùng lặp theo `(device_id, metric, measured_at)`	Spark, dbt	`row_number()` và `MERGE` theo khóa
FR4.4	Lưu vết nguồn gốc trên mọi bản ghi	Spark, Iceberg	Cột topic, partition, offset, `ingested_at`, `batch_id`
FR4.5	Tóm tắt chất lượng hằng ngày	dbt, Trino	Model gold `quality_daily`
FR4.6	Bảng điều khiển loại bỏ hoặc đánh dấu dữ liệu lỗi	FastAPI, web	Cột `quality_flag` trả về cùng giá trị
FR4.7	Test dbt chặn làm mới gold khi thất bại	dbt	`dbt build` bỏ qua model phía sau test lỗi
FR8.1	Phát hiện vượt ngưỡng theo cây trồng	Spark streaming	Ngưỡng `alert_below`, `alert_above` theo cây trồng
FR8.2	Phát hiện mất kết nối, pin yếu, sóng yếu	Spark streaming	Thời gian im lặng so với chu kỳ báo cáo
FR8.3	Lưu cảnh báo với loại, mức độ, thửa ruộng, trạng thái	Cassandra	Bảng `alerts_by_field`, `alerts_by_status`, `alert_by_id`
FR8.4	Vòng đời open, acknowledged, resolved	Spark, FastAPI	Máy trạng thái thuần trong `alerts.py`; `PATCH /alerts/{id}`
FR8.5	Chống bão cảnh báo (debounce, cooldown)	Spark, Cassandra	Bảng `alert_state_by_field`
FR8.6	Danh sách cảnh báo đang mở và lịch sử	FastAPI, web	`GET /alerts?status=`

CHƯƠNG 4

THIẾT KẾ GIẢI PHÁP VÀ HIỆN THỰC ỨNG DỤNG

4.1 Kiến trúc tổng thể

4.2 Công nghệ quản lý dữ liệu

4.3 Công nghệ xử lý dữ liệu

4.4 Thiết kế mô hình dữ liệu

4.5 Pipeline micro-batch theo từng yêu cầu nghiệp vụ

4.5.1 BR01 – Tập trung hóa giám sát nông nghiệp

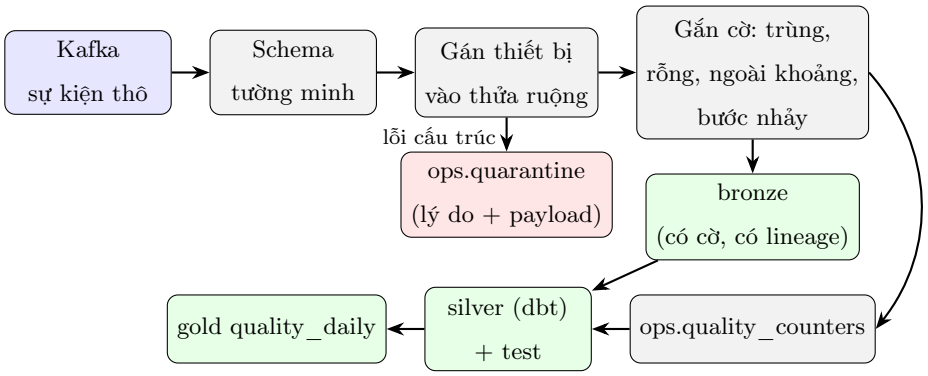
4.5.2 BR02 – Nâng cao nhận biết điều kiện đồng ruộng

4.5.3 BR03 – Duy trì tính liên tục của giám sát

4.5.4 BR04 – Thiết lập dữ liệu đáng tin cậy

Yêu cầu. Người dùng không được ra quyết định dựa trên số đo sai, trùng hoặc gây hiểu nhầm, và mọi con số phải truy ngược được về nguồn (FR4.1–FR4.7).

Luồng dữ liệu. Việc bảo đảm chất lượng được đặt ở hai nơi. Kiểm định chạy *trong* mỗi micro-batch của Spark, nên dữ liệu lỗi bị chặn trước khi tới bảng điều khiển gần thời gian thực. Tổng hợp và kiểm thử chạy *sau* trong dbt, nên tầng gold chỉ được làm mới khi dữ liệu đạt các test. Hình 4.1 mô tả luồng này.



Hình 4.1: Luồng bảo đảm chất lượng dữ liệu của BR04

Bước 1: schema tường minh (FR4.1). Tác vụ không suy diễn schema. Mọi trường của payload được đọc dưới dạng chuỗi, để phân biệt được hai trường hợp: giá trị sai kiểu (ví dụ `"value": "high"`) là lỗi cấu trúc và bị cách ly, còn giá trị `null` hợp lệ về JSON là một số đo rỗng và chỉ bị gắn cờ.

```
PAYLOAD_COLUMNS = {
    "event_id": "string", "device_id": "string", "metric": "string",
    "value": "string", "reading_ts": "string",
    "battery_pct": "string", "signal_quality": "string",
    "simulated_fault": "string",
}
schema = StructType([StructField(n, StringType(), True) for n in PAYLOAD_COLUMNS
    ])
```

Đoạn mã 4.1: Schema tường minh của payload (`src/streaming/config.py`)

Bước 2: cách ly lỗi cấu trúc. Mỗi bản ghi được gán một lý do cách ly theo thứ tự ưu tiên ở Đoạn mã 4.2. Bản ghi có lý do khác rỗng đi vào `ops.quarantine`; phần còn lại đi tiếp. Nhờ vậy một payload hỏng không làm dừng cả micro-batch.

```
reason = (
    F.when(~F.col("json_ok"), F.lit("invalid_json"))
    .when(blank_device | blank_metric | blank_ts, F.lit("missing_field"))
    .when(F.col("measured_at").isNull(), F.lit("invalid_timestamp"))
    .when(F.col("value_raw").isNotNull() & value.isNull(), F.lit("
    malformed_value"))
    .when(F.col("resolved_field_id").isNull(), F.lit("unknown_device"))
    .when(~F.array_contains(device_metrics[F.col("device_id")], F.col("metric"))
    ,
          F.lit("unexpected_metric"))
    .otherwise(F.lit(None).cast("string"))
)
```

Đoạn mã 4.2: Lý do cách ly (`src/streaming/pipeline.py`, `_stage`)

Bước 3: gắn cờ chất lượng (FR4.2, FR4.3). Với các bản ghi hợp lệ về cấu trúc, cờ `quality_flag` nhận một trong năm giá trị: `duplicate`, `null_value`, `out_of_range`, `implausible_jump` hoặc `ok`. Ngưỡng của từng chỉ số nằm trong dữ liệu danh mục (Bảng 4.1), nên đổi ngưỡng không cần sửa mã. Bước nhảy được so với giá trị mới nhất của cùng thửa ruộng và chỉ số trong Cassandra.

Bảng 4.1: Ngưỡng kiểm định theo chỉ số (`reference/reference_data.json`)

Chỉ số	Đơn vị	Nhỏ nhất	Lớn nhất	Bước nhảy tối đa
soil_moisture	%	0	100	25
soil_temperature	°C	−10	50	12
air_temperature	°C	−30	60	15
relative_humidity	%	0	100	30
rainfall	mm/h	0	200	40

Bước 4: ghi bronze, bảng đếm và lineage (FR4.4, FR4.5). Bản ghi được ghi vào bronze bằng `MERGE` theo khóa `(device_id, metric, measured_at)`, kèm sáu cột lineage. Đồng thời mỗi lô ghi một dòng đếm cho mỗi cặp thiết bị và chỉ số vào `ops.quality_counters` (tổng, hợp lệ, trùng, rỗng, ngoài khoảng, bước nhảy, đến muộn). Bảng đếm này là nguồn của báo cáo chất lượng hằng ngày.

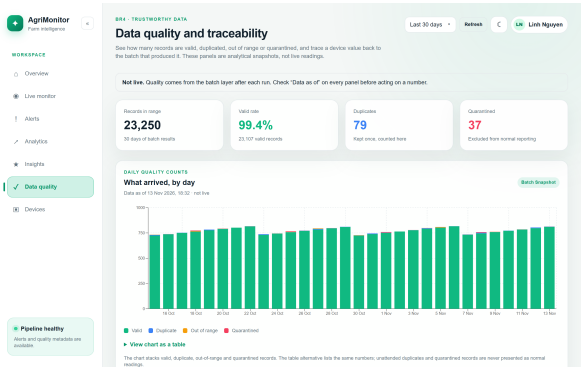
Bước 5: dbt và tầng gold (FR4.5, FR4.7). Model silver khử trùng lặp lần thứ hai (Đoạn mã 2.2) và mang các test ở Đoạn mã 2.3. Model gold `quality_daily` cộng bảng đếm theo ngày và ghép với số bản ghi bị cách ly:

```
with counters as (
    select cast(date_trunc('day', window_start) as date) as day,
           sum(total) as total, sum(valid) as valid,
           sum(duplicate) as duplicate, sum(out_of_range) as out_of_range,
           sum(null_value) as quarantined
    from {{ source('ops', 'quality_counters') }} group by 1
),
quarantine as (
    select cast(date_trunc('day', detected_at) as date) as day,
           count(*) as quarantined
    from {{ source('ops', 'quarantine') }} group by 1
)
select c.day, c.valid, c.duplicate, c.out_of_range,
       coalesce(q.quarantined, c.quarantined, 0) as quarantined, c.total
from counters c left join quarantine q on q.day = c.day
```

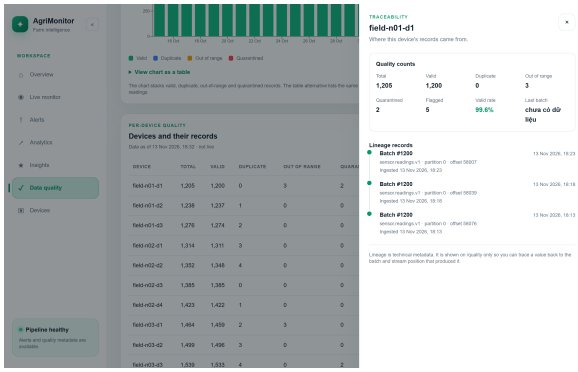
Đoạn mã 4.3: Model gold tóm tắt chất lượng (`dbt/models/gold/quality_daily.sql`, rút gọn)

Cột `quarantined` lấy số bản ghi bị cách ly trong ngày; nếu ngày đó không có bản ghi cách ly nào, model dùng số bản ghi rỗng (`null_value`) thay thế. Hai khái niệm này khác nhau (bản ghi rỗng vẫn nằm trong bronze), nên Mục 5.2 đối chiếu chúng riêng.

Bước 6: hiển thị (FR4.6). FastAPI đọc `quality_daily` và bảng silver qua Trino cho trang *Data quality*, gồm biểu đồ theo ngày và bảng theo thiết bị có xem lineage. Trên đường gần thời gian thực, mỗi giá trị trả về kèm `quality_flag` để giao diện đánh dấu điểm đáng ngờ.



(a) Biểu đồ số bản ghi theo ngày



(b) Lineage của một thiết bị

Hình 4.2: Trang chất lượng dữ liệu của ứng dụng (ảnh chụp với dữ liệu mô phỏng của chế độ `DATA_MODE=mock`).

Kết quả và hạn chế hiện tại. Mọi bản ghi lỗi đều được giữ lại với lý do hoặc cờ, và mọi dòng mang lineage. Có hai điểm còn phải sửa trước khi đánh giá ở Chương 5. Thứ nhất, model gold `field_daily` hiện tính min, max và trung bình trên mọi giá trị khác rỗng, kể cả dòng có cờ `out_of_range`; cần giới hạn ở `quality_flag = 'ok'`. Thứ hai, các model gold mới chỉ có test `not_null`; cần thêm test khoảng giá trị (accepted range) để FR4.7 có tác dụng ở tầng gold.

4.5.5 BR05 – Hỗ trợ quản lý dựa trên dữ liệu

4.5.6 BR06 – Đánh giá lịch sử và lập kế hoạch

4.5.7 BR07 – Làm cho thông tin dễ tiếp cận

4.5.8 BR08 – Hỗ trợ ứng phó sớm với rủi ro

Yêu cầu. Vận hành viên cần biết sớm khi thửa ruộng có điều kiện bất thường hoặc thiết bị mất kết nối, ghi lại việc mình đã xử lý, và không bị ngập trong cảnh báo lặp lại (FR8.1–FR8.6).

Luồng dữ liệu. Luật cảnh báo chạy ở cuối mỗi micro-batch, sau khi dữ liệu đã được kiểm định (Mục 4.5.4). Mỗi lô sinh ra một danh sách *điều kiện* (condition). Mỗi điều kiện đi qua một máy trạng thái có debounce và cooldown, rồi được ghi vào các bảng cảnh báo trong Cassandra. FastAPI đọc các bảng này và nhận thao tác acknowledge hoặc resolve từ vận hành viên.

4.5.8.1 Các loại điều kiện

Bảng 4.2 liệt kê các điều kiện được đánh giá trên mỗi lô. Ngưỡng theo cây trồng nằm trong dữ liệu danh mục; ngưỡng thiết bị lấy từ biến môi trường.

Bảng 4.2: Các điều kiện cảnh báo (`src/streaming/pipeline.py`, `_build_conditions`)

Điều kiện	**Khi nào vi phạm**	**Mức độ**	**FR**
Dưới ngưỡng cây trồng (`low_soil_moisture`, ...)	Trung bình các giá trị `ok` trong lô nhỏ hơn `alert_below` của cây trồng	warning	FR8.1
Trên ngưỡng (`heat_stress`, ...)	Trung bình lớn hơn `alert_above`	critical	FR8.1
Thay đổi đột ngột (`rapid_change`)	Có bản ghi cờ `implausible_jump`	info	FR8.1
Mất kết nối (`device_offline`)	Thiết bị im lặng lâu hơn 4 lần chu kỳ báo cáo	warning	FR8.2
Pin yếu (`battery_low`)	Pin $\leq 20\%$	warning	FR8.2
Sóng yếu (`signal_poor`)	Chất lượng tín hiệu $< 0{,}3$	info	FR8.2

Ví dụ với lúa (`RICE`): khoảng độ ẩm đất mục tiêu là 30–60% và ngưỡng cảnh báo `alert_below` là 32%. Nếu trung bình độ ẩm các bản ghi hợp lệ trong một lô của thửa F001 là 29,5%, điều kiện `low_soil_moisture` của F001 ở trạng thái vi phạm.

Điều kiện mất kết nối được đánh giá cho *mọi* thiết bị trong danh mục ở mỗi lô, kể cả khi lô không có dữ liệu. Để Spark vẫn chạy lô khi Kafka im lặng hoàn toàn, nguồn Kafka được hợp (`unionByName`) với nguồn `rate` của Spark phát một dòng nhịp mỗi giây; các dòng nhịp bị lọc bỏ ngay sau đó. Nếu thiếu cơ chế này, sự cố mất kết nối toàn bộ sẽ không bao giờ được phát hiện vì không có lô nào chạy.

4.5.8.2 Máy trạng thái: debounce, cooldown và tự đóng

Toàn bộ logic quyết định nằm trong một hàm thuần Python, không phụ thuộc Spark hay Cassandra, nên kiểm thử đơn vị được (Đoạn mã 4.4).

```python
def evaluate(condition, state, now, cfg) -> Decision:
    if condition.breaching:
        state.debounce_count += 1
        if state.open_alert_id is not None:          # already open
            return Decision(emit_alert=False, resolve_alert=False)
        if state.debounce_count < cfg.debounce_batches:   # debounce
            return Decision(emit_alert=False, resolve_alert=False)
        if state.last_emitted_at is not None and \
           (now - state.last_emitted_at).total_seconds() < cfg.cooldown_s:
            return Decision(emit_alert=False, resolve_alert=False)  # cooldown
        return Decision(emit_alert=True, resolve_alert=False)
    state.debounce_count = 0                             # condition cleared
    if state.open_alert_id is not None and state.open_status == "open":
        return Decision(emit_alert=False, resolve_alert=True)    # auto-resolve
    return Decision(emit_alert=False, resolve_alert=False)
```

Đoạn mã 4.4: Máy trạng thái cảnh báo (`src/streaming/alerts.py`, rút gọn)

Ba quy tắc chống bão cảnh báo (FR8.5) và vòng đời (FR8.4) như sau:

- **Debounce**: điều kiện phải vi phạm liên tiếp `ALERT_DEBOUNCE_BATCHES` lô (mặc định 2) mới sinh cảnh báo, nên một lô nhiễu không gây cảnh báo. Với trigger 30 giây, cảnh báo xuất hiện sau khoảng 60 giây kể từ lô vi phạm đầu tiên.
- **Một cảnh báo cho một sự cố**: khi đã có cảnh báo mở cho điều kiện đó, các lô vi phạm tiếp theo không sinh thêm cảnh báo.
- **Cooldown**: sau khi một cảnh báo được phát, cùng điều kiện không được phát lại trong `ALERT_COOLDOWN_S` giây (mặc định 300).
- **Tự đóng**: khi điều kiện hết vi phạm, cảnh báo đang ở trạng thái `open` được đóng tự động với lý do `condition_cleared`. Cảnh báo mất kết nối vì thế tự đóng khi thiết bị gửi dữ liệu trở lại. Cảnh báo đã được *acknowledge* thì không tự đóng, vì đã có người nhận xử lý và phải do họ đóng.

Trạng thái của từng cặp (thửa ruộng, điều kiện), gồm bộ đếm debounce, thời điểm phát gần nhất và cảnh báo đang mở, được lưu trong bảng Cassandra `alert_state_by_field`. Vì vậy tác vụ không cần toán tử có trạng thái của Spark, và bộ đếm vẫn còn sau khi tác vụ khởi động lại.

4.5.8.3 Lưu trữ và vòng đời trong Cassandra

Cassandra được mô hình hóa theo truy vấn, nên một cảnh báo được ghi vào ba bảng, mỗi bảng phục vụ một câu truy vấn (Bảng 4.3). Lịch sử cảnh báo được giữ 90 ngày.

Bảng 4.3: Các bảng cảnh báo trong Cassandra (`cassandra/schema.cql`)

Bảng	Khóa chính	Phục vụ truy vấn
`alerts_by_field`	`((field_id), created_at, alert_id)`	Lịch sử cảnh báo của một thửa ruộng, mới nhất trước
`alerts_by_status`	`((status), created_at, alert_id)`	Danh sách cảnh báo đang mở cho bảng điều khiển
`alert_by_id`	`((alert_id))`	Tra cứu một cảnh báo khi acknowledge hoặc resolve
`alert_state_by_field`	`((field_id), condition_code)`	Trạng thái debounce và cooldown của tác vụ streaming

Vì Cassandra không cho cập nhật cột khóa, đổi trạng thái một cảnh báo nghĩa là xóa dòng khỏi phân vùng trạng thái cũ của `alerts_by_status` và chèn vào phân vùng mới. Bảng `alerts_by_field` giữ thông tin người xử lý (`acknowledged_by`, `resolved_by`), thời điểm, cách đóng (`manual` hoặc `condition_cleared`) và ghi chú, nên lịch sử phản hồi được bảo toàn.

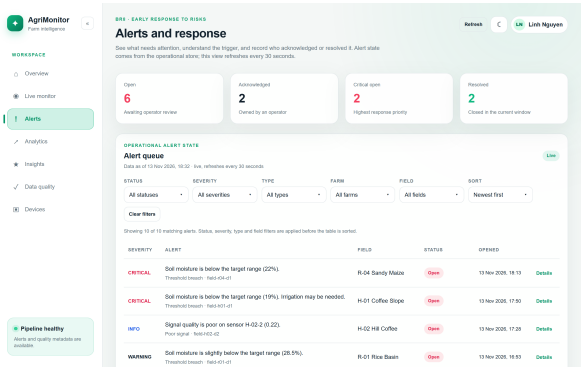
Thao tác của vận hành viên đi qua `PATCH /alerts/{id}` của FastAPI. API kiểm tra chuyển trạng thái theo cùng bảng `ALLOWED_TRANSITIONS` với tác vụ streaming và trả mã 409 khi chuyển trạng thái không hợp lệ (ví dụ acknowledge một cảnh báo đã đóng):

```
ALLOWED_TRANSITIONS = {
    "open":         {"acknowledged", "resolved"},
    "acknowledged": {"resolved"},
    "resolved":     set(),
}
```

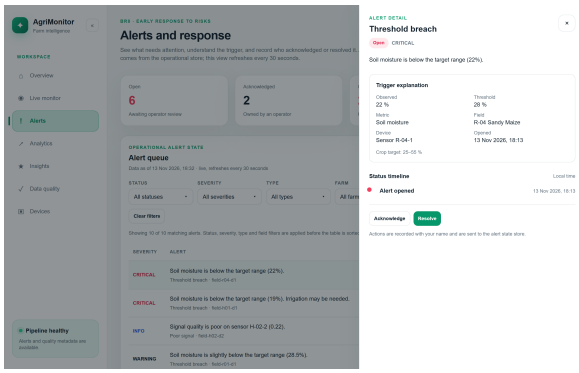
Đoạn mã 4.5: Chuyển trạng thái hợp lệ (`src/streaming/alerts.py`)

```
curl -X PATCH http://localhost:8000/alerts/<alert_id> \
     -H "content-type: application/json" \
     -d '{"action": "acknowledge", "actor": "operator-1", "note": "Irrigation
    valve opened"}'
```

Đoạn mã 4.6: Acknowledge một cảnh báo qua API



(a) Hàng đợi cảnh báo và bộ lọc



(b) Chi tiết, timeline và thao tác

Hình 4.3: Trang cảnh báo của ứng dụng (ảnh chụp với dữ liệu mô phỏng của chế độ `DATA_MODE=mock`).

Kiểm thử. Logic cảnh báo có 7 kiểm thử đơn vị trong `tests/unit/test_alerts.py`: debounce hoãn các lô đầu, một lô nhiễu không mở cảnh báo, cảnh báo đang mở không bị phát lại, điều kiện hết thì cảnh báo tự đóng, cảnh báo đã acknowledge không tự đóng, cooldown chặn cảnh báo thứ hai quá sớm, và bảng chuyển trạng thái. Logic mất kết nối và khoảng trống dữ liệu có 11 kiểm thử trong `tests/unit/test_status.py`. Cả hai chạy bằng `just test-unit`, không cần khởi động cụm.

4.6 Giải pháp thay thế để so sánh

4.7 Hiện thực ứng dụng

4.7.1 Mô phỏng và thu thập dữ liệu

4.7.2 Giao diện người dùng

4.7.3 Triển khai và cấu hình

Toàn bộ hệ thống chạy trên một máy bằng Docker Compose. Ba nguyên tắc triển khai: (1) mỗi nhóm dịch vụ khởi động được độc lập qua *profile* của Compose; (2) mọi thao tác thường ngày là một *recipe* của công cụ `just`; (3) mọi cấu hình đi qua biến môi trường, không có địa chỉ hay mật khẩu nào viết cứng trong mã.

Các profile của Docker Compose. Bảng 4.4 liệt kê bốn profile trong `docker-compose.yml`. Cassandra thuộc cả hai profile `processing` và `serving`, vì cả tác vụ streaming lẫn API đều cần nó.

Bảng 4.4: Các profile Docker Compose và dịch vụ (`docker-compose.yml`)

Profile	Dịch vụ	Vai trò
`core`	namenode, datanode, kafka, ingest, hive-metastore, trino	Lưu trữ, hàng đợi sự kiện, catalog và engine truy vấn
`processing`	spark-stream, spark-batch, dbt, spark-tests, simulator, cassandra	Xử lý streaming, xử lý theo lô, kiểm thử, sinh dữ liệu
`serving`	cassandra, cassandra-init, api	Kho phục vụ gần thời gian thực và FastAPI
`app`	web	Ứng dụng web Next.js

Phiên bản thành phần. Bảng 4.5 ghi phiên bản các ảnh Docker được dùng. Mục 5.1 dùng lại bảng này để mô tả môi trường thực nghiệm.

Bảng 4.5: Phiên bản các thành phần

Thành phần	Phiên bản	Thành phần	Phiên bản
Apache Hadoop (HDFS)	3.3.6	Apache Iceberg	1.7.1
Apache Kafka (KRaft)	3.8.0	Hive Metastore	3.1.3
Apache Spark	3.5.3	Trino	438
Apache Cassandra	5.0.3	Python	3.11

Các recipe `just`. Người dùng không cần nhớ lệnh `docker compose` dài. Đoạn mã 4.7 là trình tự khởi động một phiên demo đầy đủ.

```
just init             # create .env from .env.example (never overwrites)
just doctor           # check just, docker, compose and .env
just up               # core + serving + processing profiles
just run-stream-bg    # Spark micro-batch job, 30 s trigger
just run-simulator    # LoRaWAN-style sensors with fault injection
just batch-run        # lakehouse-init, dbt build, dbt test, compaction
just up-app           # FastAPI + Next.js web app
just verify-pipeline # row counts in Cassandra, bronze and quarantine
```

Đoạn mã 4.7: Trình tự khởi động hệ thống (`justfile`)

Bảng 4.6 nhóm các recipe theo mục đích.

Bảng 4.6: Các recipe `just` chính

Nhóm	**Recipe**
Vòng đời	`up`, `up-core`, `up-serving`, `up-processing`, `up-app`, `down`, `clean`, `ps`, `logs`
Streaming	`run-stream`, `run-stream-bg`, `stop-stream`, `streaming-reset`
Dữ liệu vào	`run-simulator`, `sim-for`, `ingest-check`, `kafka-tail`
Theo lô	`lakehouse-init`, `dbt-run`, `dbt-test`, `dbt-docs`, `compact`, `batch-run`
Kiểm tra	`verify-pipeline`, `trino-sql`, `api-health`, `test`, `test-unit`, `lint`
Benchmark	`bench-trigger`, `bench-lookups`, `bench-storage`, `bench-compaction`, `bench-replay`

Cấu hình qua biến môi trường. Tệp `.env.example` là mẫu cấu hình duy nhất; `just init` sao chép nó thành `.env`, tệp này không được commit. Các tham số của tác vụ streaming được truyền vào container qua khối `x-spark-env` của Compose, với giá trị mặc định đặt ngay tại đó (Đoạn mã 4.8). Nhờ vậy benchmark có thể đổi chu kỳ trigger mà không sửa mã, ví dụ `just run-stream trigger=10s`.

```
SPARK_DRIVER_MEMORY: ${SPARK_DRIVER_MEMORY:-2g}
TRIGGER_INTERVAL: ${TRIGGER_INTERVAL:-30s}
MAX_OFFSETS_PER_TRIGGER: ${MAX_OFFSETS_PER_TRIGGER:-20000}
WATERMARK_S: ${WATERMARK_S:-3600}
LATE_AFTER_S: ${LATE_AFTER_S:-600}
ALERT_DEBOUNCE_BATCHES: ${ALERT_DEBOUNCE_BATCHES:-2}
ALERT_COOLDOWN_S: ${ALERT_COOLDOWN_S:-300}
DEVICE_OFFLINE_MULTIPLIER: ${DEVICE_OFFLINE_MULTIPLIER:-4}
BATTERY_LOW_PCT: ${BATTERY_LOW_PCT:-20}
SIGNAL_POOR_MAX: ${SIGNAL_POOR_MAX:-0.3}
```

Đoạn mã 4.8: Tham số streaming và cảnh báo (`docker-compose.yml`, khối `x-spark-env`, rút gọn)

Yêu cầu tài nguyên. Docker cần tối thiểu 8 GB bộ nhớ; 12 GB thoải mái hơn khi HDFS, Kafka, Cassandra, Spark, Trino và Hive Metastore chạy cùng lúc. Spark chạy ở chế độ `local[*]` với bộ nhớ driver 2 GB, 4 nhân và 4 phân vùng shuffle, phù hợp khối lượng dữ liệu demo.

CHƯƠNG 5

ĐÁNH GIÁ

5.1 Thiết lập thực nghiệm

Mọi kết quả trong chương này được đo trên cùng một môi trường. Mỗi bảng kết quả ở các mục sau chỉ ghi các tham số khác với mục này.

5.1.1 Phần cứng và phần mềm

Bảng 5.1: Môi trường thực nghiệm

Hạng mục	**Giá trị**
CPU	[Mẫu CPU, số nhân, số luồng]
RAM của máy	[Dung lượng RAM]
Bộ nhớ cấp cho Docker	[Ví dụ 12 GB]
Ổ đĩa	[SSD hoặc HDD, dung lượng trống]
Hệ điều hành	[Ví dụ Windows 11 + WSL2, hoặc macOS]
Docker Engine, Compose	[Phiên bản, lấy từ `just doctor`]
Thành phần hệ thống	Theo Bảng 4.5
Spark	`local[*]`, driver 2 GB, 4 nhân, 4 phân vùng shuffle

5.1.2 Dữ liệu thực nghiệm

Dữ liệu được sinh bởi bộ mô phỏng cảm biến (Mục 4.7) dựa trên dữ liệu danh mục `reference/reference_data.json`: 4 trang trại, 8 thửa ruộng, 24 thiết bị, 4 loại cây trồng (lúa, ngô, cà chua, cà phê) và 5 chỉ số. Mỗi thiết bị báo cáo 5 giây một lần. Bộ mô phỏng có hạt giống cố định, nên cùng cấu hình sinh ra cùng chuỗi sự kiện.

Bộ mô phỏng tiêm lỗi có chủ đích và đánh dấu mỗi sự kiện lỗi bằng trường `simulated_fault`. Trường này đi theo bản ghi vào bronze (và nằm trong `raw_payload` của bản ghi bị cách ly), nên có thể so số lỗi tiêm vào với số lỗi hệ thống phát hiện được mà không cần nhãn bên ngoài.

Bảng 5.2: Cấu hình bộ mô phỏng (`.env.example`)

Tham số	**Mặc định**	**Ý nghĩa**
`SIM_SEED`	42	Hạt giống ngẫu nhiên
`SIM_RATE_HZ`	1	Số nhịp sinh dữ liệu mỗi giây
`SIM_FAULT_RATE`	0,05	Xác suất ở mỗi nhịp có một thiết bị ngẫu nhiên được lên lịch tiêm lỗi
`SIM_FAULTS`	7 loại	`null_value`, `out_of_range`, `implausible_jump`, `duplicate`, `unknown_device`, `unexpected_metric`, `malformed`
`SIM_GAP_RATE`	0,002	Xác suất ở mỗi nhịp có một thiết bị ngẫu nhiên bắt đầu im lặng
`SIM_GAP_SECONDS_MIN/MAX`	45 / 90	Độ dài khoảng im lặng (giây)
`SIM_DURATION_SECONDS`	[giá trị dùng khi đo]	Thời gian chạy một lần thực nghiệm

5.1.3 Tham số của đường xử lý

Bảng 5.3: Tham số mặc định của tác vụ streaming và cảnh báo

Tham số	Giá trị	Ý nghĩa
TRIGGER_INTERVAL	30 s	Chu kỳ micro-batch (benchmark #1 đổi thành 10 s và 60 s)
MAX_OFFSETS_PER_TRIGGER	20 000	Số sự kiện tối đa mỗi lô
WATERMARK_S	3 600	Cửa sổ chấp nhận dữ liệu đến muộn
LATE_AFTER_S	600	Ngưỡng gắn cờ `is_late`
ALERT_DEBOUNCE_BATCHES	2	Số lô vi phạm liên tiếp trước khi cảnh báo
ALERT_COOLDOWN_S	300	Khoảng tối thiểu giữa hai cảnh báo cùng điều kiện
DEVICE_OFFLINE_MULTIPLIER	4	Im lặng quá 4 lần chu kỳ báo cáo (20 s) thì coi là mất kết nối
BATTERY_LOW_PCT	20	Ngưỡng pin yếu (%)
SIGNAL_POOR_MAX	0,3	Ngưỡng sóng yếu

5.1.4 Quy trình chạy lại

Mỗi thực nghiệm bắt đầu từ trạng thái sạch để kết quả lặp lại được:

```
just clean && just up          # empty HDFS, Kafka, Cassandra
just run-stream-bg             # start the micro-batch job
just sim-for <seconds>         # fixed-length simulated load
just batch-run                 # dbt build + tests + compaction
just verify-pipeline           # counts used by the evaluation
```

Đoạn mã 5.1: Quy trình chuẩn của một lần thực nghiệm

Kết quả của từng benchmark được lưu trong `benchmarks/results/*.csv`, với cấu hình của lần đo ghi ở các dòng đầu tệp.

5.2 Tính đúng đắn của dữ liệu

Mục này kiểm tra dữ liệu sau khi qua đường xử lý có đúng hay không, theo bốn câu hỏi:

1. Mọi lỗi được tiêm vào có bị phát hiện và đếm đúng không?
2. Tầng gold có khớp với tầng silver không?

3. Tầng gold có chứa giá trị phi lý không?
4. Sau khi tác vụ bị dừng và khởi động lại, có mất hoặc trùng dữ liệu không?

Mọi lần đo dùng môi trường và quy trình ở Mục 5.1.

5.2.1 Phát hiện lỗi được tiêm vào

Phương pháp. Bộ mô phỏng gắn nhãn `simulated_fault` cho mỗi sự kiện lỗi. Mỗi loại lỗi có một kết quả mong đợi: hoặc bị cách ly với một lý do cụ thể, hoặc được giữ lại với một cờ chất lượng cụ thể (Bảng 5.4). Số lỗi tiêm vào được đếm từ nhãn; số lỗi phát hiện được đếm từ `ops.quarantine` và `ops.quality_counters`. Hai số phải bằng nhau.

Bảng 5.4: Kết quả mong đợi cho từng loại lỗi được tiêm vào

Lỗi tiêm vào	**Cách bộ mô phỏng tạo lỗi**	**Kết quả mong đợi**
`null_value`	Giá trị đo là `null`	Cờ `null_value`
`out_of_range`	Giá trị lớn hơn ngưỡng tối đa từ 5 đến 50 đơn vị	Cờ `out_of_range`
`implausible_jump`	Giá trị lệch 3–10 lần bước nhảy tối đa	Cờ `implausible_jump`
`duplicate`	Gửi lại nguyên một payload đã gửi trước đó	Cờ `duplicate`, chỉ một bản vào bronze
`unknown_device`	Mã thiết bị không có trong danh mục	Cách ly, lý do `unknown_device`
`unexpected_metric`	Tên chỉ số sai (`..._typo`)	Cách ly, lý do `unexpected_metric`
`malformed`	JSON hỏng, thiếu trường, thời gian sai, giá trị không phải số	Cách ly, lý do `invalid_json`, `missing_field`, `invalid_timestamp` hoặc `malformed_value`

Có ba điểm cần lưu ý khi đếm:

- **Đếm bản sao.** Với lỗi `duplicate`, bộ mô phỏng giữ payload ở lần đầu thiết bị bị chọn và chỉ gửi lại ở lần thứ hai. Số bản sao thật vì vậy phải đếm bằng số `event_id` lặp lại, không đếm bằng số nhãn.
- **Bản sao đến ở lô khác.** Cờ `duplicate` được gán bằng `row_number()` *trong một lô*. Nếu bản sao đến ở một lô sau, nó mang cờ `ok` và lệnh `MERGE` gộp nó vào dòng đã có. Bronze vẫn không bị trùng, nhưng bảng đếm sẽ ghi thiếu số bản sao. Vì bản sao của bộ mô phỏng thường cách bản gốc nhiều lô, đây là nguồn lệch được dự đoán của dòng `duplicate` trong Bảng 5.5.

- **Bước nhảy.** Bước nhảy được so với giá trị mới nhất của cả thửa ruộng, nên một giá trị bước nhảy đến ngay sau một giá trị ngoài khoảng có thể không bị coi là bước nhảy.

Câu truy vấn 5.2 đếm cả hai phía trên Trino.

```
-- Faults kept in bronze: injected label vs quality flag
SELECT simulated_fault, quality_flag, count(*) AS n
FROM lakehouse.bronze.sensor_readings
WHERE simulated_fault IS NOT NULL
GROUP BY 1, 2 ORDER BY 1, 2;

-- Faults sent to quarantine: injected label (from the raw payload) vs reason
SELECT json_extract_scalar(raw_payload, '$.simulated_fault') AS injected,
       reason, count(*) AS n
FROM lakehouse.ops.quarantine
GROUP BY 1, 2 ORDER BY 1, 2;

-- Duplicates removed before bronze
SELECT sum(duplicate) FROM lakehouse.ops.quality_counters;
```

Đoạn mã 5.2: Đếm lỗi tiêm vào và lỗi phát hiện trên Trino

Kết quả. Bảng 5.5 là kết quả của một lần chạy [thời lượng] giây với cấu hình mặc định.

Bảng 5.5: Số lỗi tiêm vào so với số lỗi phát hiện được

Loại lỗi	Tiêm vào	Phát hiện	Tỷ lệ
null_value	[]	[]	[]
out_of_range	[]	[]	[]
implausible_jump	[]	[]	[]
duplicate	[]	[]	[]
unknown_device	[]	[]	[]
unexpected_metric	[]	[]	[]
malformed	[]	[]	[]
Tổng	[]	[]	[]

[Nhận xét kết quả: loại nào đạt 100%, loại nào lệch và vì sao.]

5.2.2 Đối soát tầng gold với tầng silver

Phương pháp. Chọn ngẫu nhiên [số] cặp (thửa ruộng, ngày), tính lại min, max, trung bình và số bản ghi trực tiếp từ silver, rồi so với `gold.field_daily`. Truy vấn 5.3 trả về các dòng lệch; kết quả mong đợi là rỗng.

```
WITH recomputed AS (
    SELECT field_id, metric, CAST(date_trunc('day', measured_at) AS date) AS day
    ,
           min(value) AS min_value, max(value) AS max_value,
           avg(value) AS avg_value, count(*) AS reading_count
    FROM lakehouse.silver.sensor_readings
    WHERE value IS NOT NULL
    GROUP BY 1, 2, 3
)
SELECT g.field_id, g.metric, g.day
FROM lakehouse.gold.field_daily g
JOIN recomputed r USING (field_id, metric, day)
WHERE g.reading_count <> r.reading_count
   OR abs(g.avg_value - r.avg_value) > 1e-9
   OR g.min_value <> r.min_value OR g.max_value <> r.max_value;
```

Đoạn mã 5.3: Đối soát gold với silver trên Trino

Kết quả. [Số cặp kiểm tra, số dòng lệch (mong đợi 0).]

5.2.3 Giá trị phi lý trong tầng gold

Phương pháp. Đếm số dòng của `gold.field_daily` có `min_value` hoặc `max_value` nằm ngoài khoảng vật lý của chỉ số (Bảng 4.1).

Kết quả. Như đã nêu ở Mục 4.5.4, phiên bản hiện tại của `field_daily` tổng hợp mọi giá trị khác rỗng, kể cả giá trị có cờ `out_of_range`, nên lần đo này *được dự đoán* sẽ tìm thấy giá trị phi lý. [Số dòng phi lý trước khi sửa.] Sau khi giới hạn model ở `quality_flag = 'ok'` và thêm test khoảng giá trị, [số dòng phi lý sau khi sửa (mong đợi 0) và log `just dbt-test`.]

5.2.4 Không mất, không trùng khi khởi động lại

Kiểm tra này dùng kết quả của benchmark #5 (Mục 5.3): dừng tác vụ streaming vài phút trong khi bộ mô phỏng vẫn gửi dữ liệu, khởi động lại, rồi so số dòng và số khóa `(device_id, metric, measured_at)` trùng lặp trong bronze và Cassandra trước và sau. Cơ chế bảo đảm là checkpoint của Spark (Mục 2.4.3) cùng các lệnh `MERGE` theo khóa (Mục 2.6.6).

```sql
SELECT device_id, metric, measured_at, count(*) AS n
FROM lakehouse.bronze.sensor_readings
GROUP BY 1, 2, 3 HAVING count(*) > 1;
```

Đoạn mã 5.4: Kiểm tra khóa trùng trong bronze

Kết quả. [Số dòng trước và sau, số khóa trùng (mong đợi 0), lấy từ `benchmarks/results/replay.cs`

5.2.5 Kiểm thử đơn vị

Ngoài các kiểm tra trên dữ liệu thật, logic quyết định được kiểm thử độc lập với Spark và Cassandra bằng `just test-unit`: 7 kiểm thử cho máy trạng thái cảnh báo và 11 kiểm thử cho logic mất kết nối và khoảng trống dữ liệu. [Kết quả lần chạy: số kiểm thử đạt / tổng.]

5.3 Hiệu năng của giải pháp

5.4 Hiệu quả hỗ trợ khai thác dữ liệu qua ứng dụng

5.4.1 BR04 – Dữ liệu đáng tin cậy

Bảng 5.6 đối chiếu các tiêu chí chấp nhận của BR4 trong tài liệu yêu cầu với bằng chứng thu được.

Bảng 5.6: Tiêu chí chấp nhận của BR4 và bằng chứng

Tiêu chí chấp nhận	**Bằng chứng**	**Kết quả**
Các lỗi do bộ mô phỏng tiêm vào (trùng, đột biến, payload hỏng, thiết bị lạ) đều được phát hiện và đếm trong báo cáo chất lượng	Bảng 5.5; trang *Data quality* hiển thị cùng số liệu	[Đạt / Chưa đạt]
Mọi con số trên bảng điều khiển truy ngược được về offset nguồn và lô	Cột lineage trên mọi dòng bronze và silver; bảng lineage theo thiết bị trên trang *Data quality* (Hình 4.2)	[Đạt / Chưa đạt]
Test dbt thất bại chặn làm mới gold (FR4.7)	Log `just dbt-run` khi cố ý làm hỏng một test silver: các model gold bị bỏ qua	[Đạt / Chưa đạt]
Bảng điều khiển loại bỏ hoặc đánh dấu dữ liệu lỗi (FR4.6)	Giá trị gần thời gian thực kèm `quality_flag`; điểm đáng ngờ được đánh dấu trên biểu đồ	[Đạt / Chưa đạt]

Kịch bản minh họa. Người dùng mở trang *Data quality*, chọn khoảng 7 ngày và thấy số bản ghi hợp lệ, trùng, ngoài khoảng và bị cách ly theo từng ngày. Khi chọn một thiết bị, bảng lineage hiện topic, phân vùng, offset, mã lô và thời điểm nhận của các bản ghi mới nhất. [Ảnh chụp màn hình và số liệu của lần demo.]

5.4.2 BR08 – Ứng phó sớm với rủi ro

Bảng 5.7: Tiêu chí chấp nhận của BR8 và bằng chứng

Tiêu chí chấp nhận	Bằng chứng	Kết quả
Một lần vượt ngưỡng mô phỏng tạo đúng một cảnh báo, trong một đến hai chu kỳ trigger cộng thời gian debounce	Số cảnh báo trong `alerts_by_field` cho điều kiện đó; thời điểm cảnh báo trừ thời điểm của số đo vi phạm đầu tiên	[Số cảnh báo, độ trễ (giây)]
Một lần mất kết nối mô phỏng tạo đúng một cảnh báo và cảnh báo tự đóng khi có dữ liệu lại	Cảnh báo `device_offline` với `resolution = condition_cleared`; bản ghi khoảng trống trong `ops.device_gaps`	[Đạt / Chưa đạt]
Vận hành viên acknowledge và resolve cảnh báo qua API, lịch sử được giữ	`PATCH /alerts/{id}`; các cột `acknowledged_by`, `resolved_by`, `note` trong `alerts_by_field`	[Đạt / Chưa đạt]
Chuyển trạng thái không hợp lệ bị từ chối	API trả 409 khi acknowledge một cảnh báo đã đóng	[Đạt / Chưa đạt]
Không có bão cảnh báo khi điều kiện kéo dài	Số cảnh báo cho một điều kiện vi phạm liên tục trong [số] phút	[Số cảnh báo (mong đợi 1)]

Cách đo độ trễ cảnh báo. Với trigger 30 giây và debounce 2 lô, độ trễ lý thuyết từ số đo vi phạm đầu tiên đến khi cảnh báo xuất hiện nằm trong khoảng 30–90 giây cộng thời gian xử lý một lô. Benchmark #1 đo giá trị này với trigger 10, 30 và 60 giây. [Kết quả đo.]

Kịch bản minh họa. Độ ẩm đất của một thửa lúa được đưa xuống dưới ngưỡng 32% và một thiết bị được cho im lặng ([cách tạo vi phạm trong demo, ví dụ tệp phát lại `SIM_FIXTURE`]). Sau khoảng một phút, trang *Alerts* hiện hai cảnh báo. Vận hành viên acknowledge cảnh báo độ ẩm kèm ghi chú, rồi resolve; cảnh báo mất kết nối tự đóng khi thiết bị gửi dữ liệu trở lại. [Ảnh chụp timeline của cảnh báo.]

5.5 So sánh với giải pháp thay thế

5.6 Thảo luận

CHƯƠNG 6

KẾT LUẬN

6.1 Kết quả đạt được

6.2 Hạn chế

6.3 Hướng phát triển

Tài liệu tham khảo

[1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st. Sebastopol, CA, USA: O'Reilly Media, 2017.

[2] Conduktor, *Micro-batching: Near-real-time stream processing*, Conduktor Glossary, Truy cập: 10/2026. [Online]. Available: `https://www.conduktor.io/glossary/micro-batching-streaming`

[3] Apache Software Foundation, *Apache kafka documentation.* [Online]. Available: `https://kafka.apache.org/documentation/`

[4] Apache Software Foundation, *Structured streaming programming guide*, Apache Spark Documentation. [Online]. Available: `https://spark.apache.org/docs/latest/streaming/`

[5] Databricks, *What is the medallion lakehouse architecture?* Truy cập: 11/2026. [Online]. Available: `https://docs.databricks.com/aws/en/lakehouse/medallion`

[6] dbt Labs, *Dbt developer hub: Models, tests and the dbt build command*, Truy cập: 11/2026. [Online]. Available: `https://docs.getdbt.com/`